

[< Go to the original](#)



Coachella Tweet Sentiment Analysis (Deep Learning Tutorial)

Fine-tuning RoBERTa base model for sentiment analysis on the Coachella 2015 dataset



Follow

 Level Up Coding a11y-light ~28 min read · June 16, 2025 (Updated: June 18, 2025) · Free: No

Are you a fan of music concerts? Coachella, one of the world's most iconic music and arts festivals, recently wrapped up its two weekend festival of music and desert vibes at the Empire Polo Club in Indio, California. It's known for featuring some of the biggest top-charting artists from genres such as hip-hop, pop, indie, rock and EDM.

What if we could measure how all the fans *feel* about the event?

Every year, after the lineup drops, fans flood social media to tweet their thoughts, feelings, and opinions leading up to and during the event. Platforms like twitter (now X) become a goldmine of public opinion. By analyzing these tweets, we can tell whether fans are excited, disappointed or just making general comments.

This is known as **sentiment analysis**, a branch of Natural Language Processing (NLP), and is widely used in domains like marketing, hospitality, social media analytics and entertainment to better understand how people *feel* about products, services, brands and events.

about the Coachella festival. We'll use the **RoBERTa-base model** from the **Hugging Face Transformers library**, built on **PyTorch framework**, and train it all for free using **Google Colab**.

Table of Contents

· [Natural Language Processing](#) · [Sentiment Analysis](#) · [Deep Learning](#) · [Transformers](#) · [Open Google Colab](#) · [Connect to a Hosted Runtime](#) · [Why is the GPU important?](#) · [How to enable GPU in Google Colab](#) · [Install and Import Libraries](#) · [Connect to Google Drive](#) · [Load Dataset](#) · [Data Understanding](#) · [Data Preparation](#) · [Exploratory Data Analysis](#) · [Sentiment Label Distribution \(Pie Chart\)](#) · [Most Common Words \(Word Cloud\)](#) · [Tweet Length Distribution \(Histogram\)](#) · [Modelling](#) · [Train-Test Split](#) · [Login to Hugging face Hub](#) · [Tokenization](#) · [Balancing Target Classes](#) · [Initialize pre-trained model](#) · [Configure training parameters and evaluation metric](#) · [Define Custom Trainer](#) · [Training and Evaluation](#) · [Training and Evaluation Summary](#) · [Push model to Hugging Face Hub](#) · [Inferencing](#) · [Load the Fine-Tuned Model from Hugging Face Hub](#) · [Preprocess Input Tweets](#) · [Sample a Real Tweet](#) · [Predict Sentiment](#) · [Deploying Gradio App](#) · [Final Thoughts](#) · [References](#)

Natural Language Processing

Natural Language Processing (NLP) is the super cool field in **Artificial Intelligence** that focuses on enabling computers and humans to communicate with natural language.

When you open a chatbot like chatGPT, you are interacting with a language model that can understand the context of your questions and answer you in natural language, like how you would speak to people in your everyday life.

Large Language Models like GPT make you feel like they are sentient but they aren't. They have been trained with a lot of **unstructured data** — Data that doesn't fit neatly into a traditional database, such as text (tweets, news articles etc.), images, audio, and video — to be able to understand patterns and the context of human language.

NLP is behind many modern applications like:

- Chatbots (e.g. chatGPT)
- Virtual assistants (e.g. Siri, Alexa)
- Language Translation

- Speech Recognition

- Auto-generated video captions... and many more

The field of NLP consists of a wide array of techniques that are aimed understanding and responding to text or voice data the same way humans do.

In this project we are focusing on sentiment analysis.

Sentiment Analysis

How exactly does a computer get to understand what your words mean. After all, computers process everything as 0's and 1's.

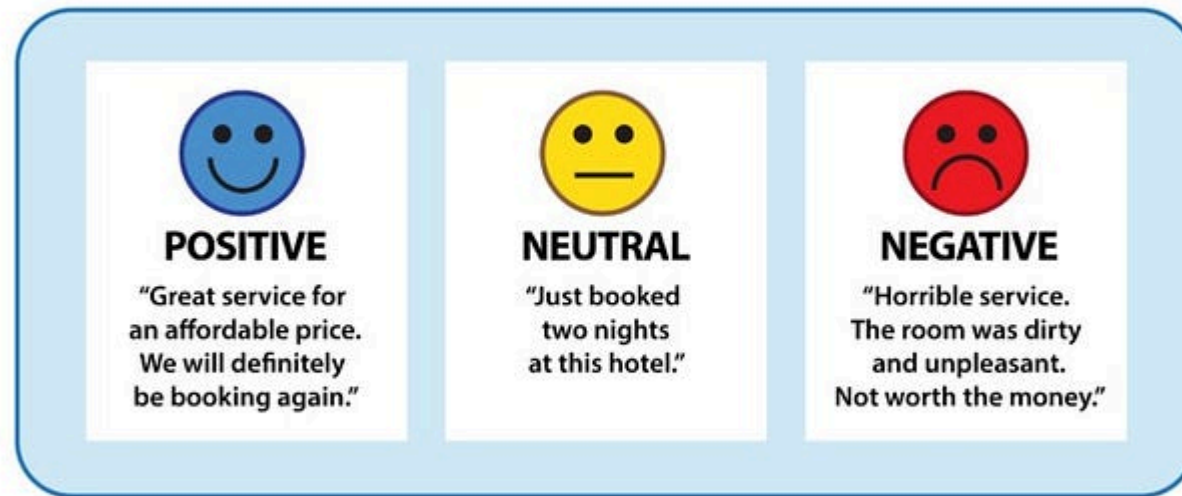
Sentiment analysis, also called opinion mining, is a popular technique in Natural Language Processing (NLP) that helps us classify opinions as positive, negative, or neutral from text data. This allows us to determine the emotional tone of text.

Sentiment analysis is widely used in tasks such as:

- Customer Experience (CX) — Classifying customer reviews
- Brand Monitoring & Reputation

We will be focusing on **categorical sentiment analysis**, where the input into a sentiment classification model is a piece of text, and the output is the probability that the sentiment expressed is **positive, negative or neutral**.

SENTIMENT ANALYSIS



Given text, sentiment analysis classifies its emotional quality.

Categorical Sentiment Analysis in NLP

sentiment analysis). Other types of sentiment analysis include:

- **Fine-grained sentiment analysis** — Goes beyond 3 categories (e.g. 5-star rating systems usually found in e-commerce stores)
- **Aspect-based sentiment analysis** — Focus on particular aspects of a product or service.

There is even a branch of sentiment analysis called toxicity classification that specifically deals with detecting hate speech, threats, and offensive comments to improve the quality of online conversations especially on some social media platforms.

Sentiment analysis can be done with traditional machine learning.

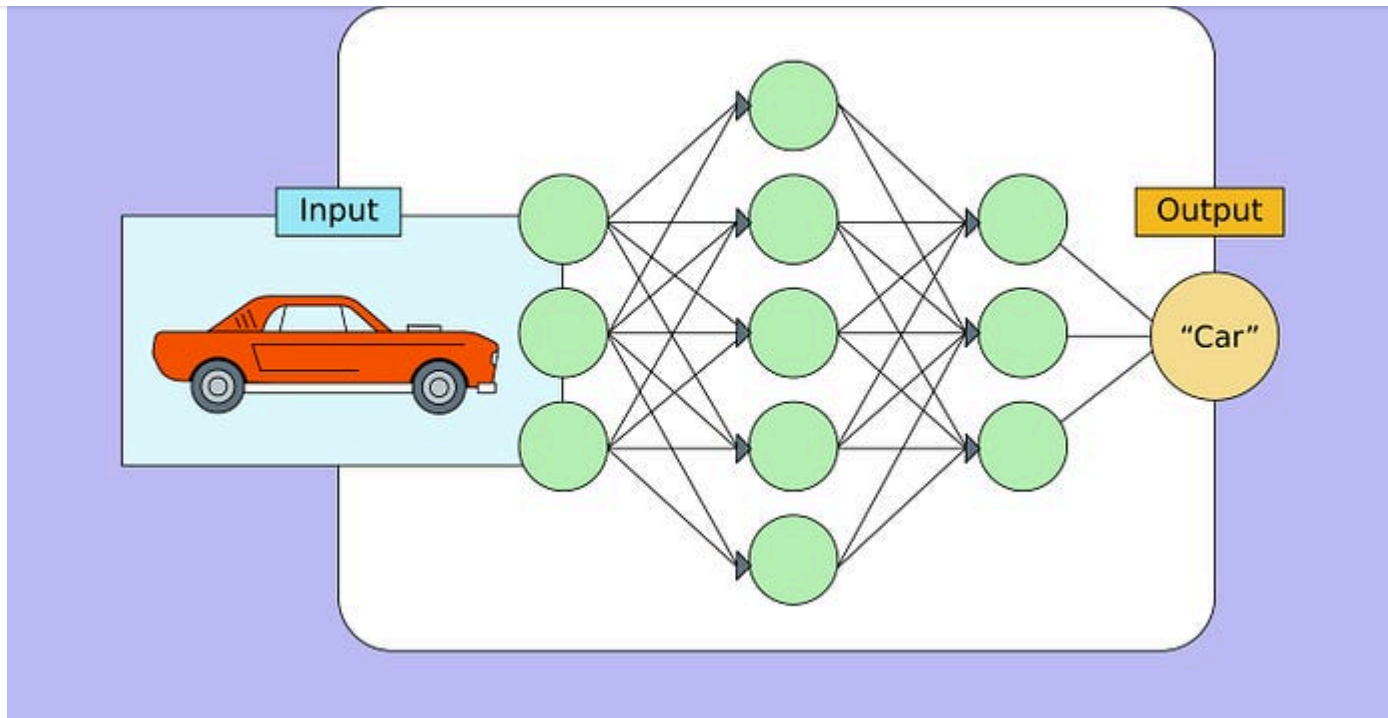
We would train a model, like logistic regression or Naive Bayes, on labeled data (in this case, tweets paired with their sentiment) and the model "learns" patterns from the training data, such as the words that appear in the text and the order in which they appear, to predict the sentiment of new, unseen text.

Classic machine learning models can do a decent job for structured or simple language, like survey responses with clear sentiment cues.

to capture much more subtle patterns and context in the language.

Deep Learning

Deep learning is a specific branch of machine learning that uses **Artificial Neural Networks** to model and solve complex problems. This is similar to how the human brain works, by processing information through layers of neurons.



What are deep neural networks?

Artificial neural networks (ANNs), also called **deep neural networks**, consist of interconnected nodes (neurons) arranged in layers.

Each neuron performs a simple mathematical operation on its input (e.g. text), passing it to other neurons in the next layer to learn complex, non-linear patterns of the data, like the kind found in natural language, and finally an output is generated (classification or a prediction).

- Sentence structure
- Word meanings and relationships
- Subtle shifts in tone and emotion

But traditional deep learning architecture, such as Recurrent Neural Networks (RNNs), has one big weakness: It process text in sequence and can't understand the full context of a sentence at once. Convolutional Neural Networks (CNNs) works well for images but lacks flexibility for languages.

That changed when Google introduced the transformer model in 2017.

Transformers

Transformers are completely running the field of AI at the moment. The transformer model emerged as a response to several limitations of earlier deep learning models, particularly in the field of Natural Language Processing (NLP).

The transformer architecture is a type of deep learning architecture that can, instead of reading sentences word by word in sequence, use a mechanism called

This enables the model to capture relationships between elements far apart in the input sequence and therefore understand the broader context of the input data much better.

Popular deep learning transformer models include:

- **GPT (Generative Pre-trained Transformer)** — Developed by OpenAI, used for text generation and language modeling. The model behind ChatGPT.
- **BERT (Bidirectional Encoder Representations from Transformers)** — Developed by Google, used for a wide range of NLP tasks. It is also the basis for the Google search engine.
- **RoBERTa** (a robustly optimized BERT variant, which we'll use)

Let's go ahead and dive into the project and find out more as we go on.

Open Google Colab

[Google Colab](#) is a free, cloud-based jupyter notebook service provided by Google that lets you write and run Python code in your web browser. You don't need to

Colab is especially popular among data scientists and machine learning practitioners because it comes preloaded with popular libraries like:

- NumPy
- Pandas
- TensorFlow
- PyTorch and others.

Plus, it offers **free access to GPUs and TPUs**, which is a big deal when training deep learning models.

Lets get started!

👉 You can open [my Colab Notebook here](#)

Connect to a Hosted Runtime

When you open a Colab notebook, you have the option to connect to a runtime(hosted, local or custom) and choose a runtime type(CPU, GPU or TPU).

speed up training.

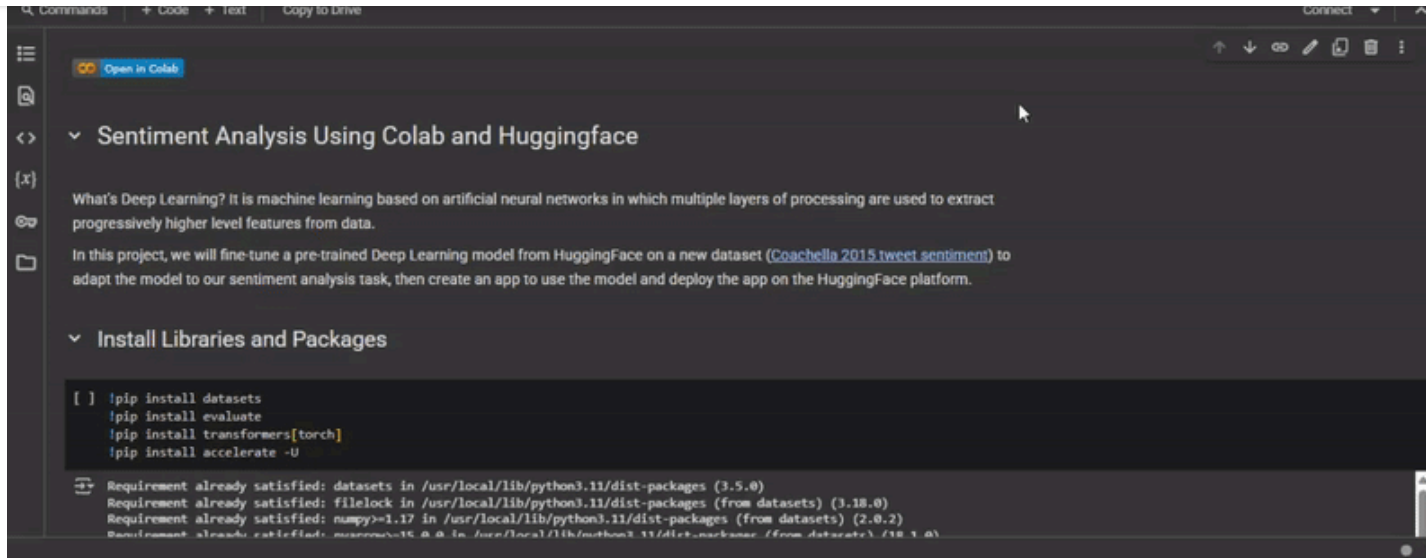
Why is the GPU important?

Fine tuning deep learning models is computationally intensive. CPUs are general-purpose processors and aren't optimized for this kind of workload.

GPUs (Graphics Processing Units), are specifically built to handle thousands of matrix operations and parallel computations at once, making them ideal for training deep learning models. Without a GPU, training could take hours or even days but with a GPU, you can cut that down to just a few minutes.

And what about **TPUs (Tensor Processing Units)**? These are custom hardware built by Google specifically for deep learning tasks, especially for TensorFlow-based models. They can offer even faster performance than GPUs for certain tasks but a GPU is more than enough for this project

How to enable GPU in Google Colab



Connect to GPU runtime

1. Click the dropdown button next to "Connect" at the top-right.
2. Choose "Change runtime type."
3. Under "Hardware accelerator," select GPU (you can also try TPU, but it's not necessary here).
4. Click "Save."
5. Now click the "Connect" button at the top-right to launch your hosted runtime.

training deep learning models in Colab. It's well-optimized, widely supported, and perfectly suited for most educational and mid-size projects.

Install and Import Libraries

First we have to install the necessary packages we need. As mentioned earlier, most libraries already come pre-installed into Colab, so we only need to install a few additional packages.

Copy

```
!pip install datasets  
!pip install evaluate  
!pip install transformers[torch]  
!pip install accelerate -U
```

We are using pip (the package installer for python) to install the following:

- `datasets` — Hugging Face library for loading, processing, and managing datasets
- `evaluate` — Library that provides tools for evaluating the performance of machine learning models using various metrics (very important)

- `accelerate` — Library for simplifying training and inference on various hardware like GPUs. The `-U` ensures it upgrades to the latest version if it's already installed, which helps avoid dependency issues (I experienced some).

After successful installation, we now import all the necessary libraries and modules that will be used throughout the project.

Copy

```
#System and data handling
import os
import re
import warnings
import pandas as pd
import numpy as np

#Data Analysis & Preparation
from evaluate import load
from collections import Counter
from datasets import load_dataset

#Scikit-Learn
from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight

#Google Drive
```

#VISUALIZATION

```
import matplotlib.pyplot as plt
from wordcloud import WordCloud
```

#Transformers

```
from transformers import AutoTokenizer, AutoConfig, AutoModelForSequenceClassification
```

#Scores

```
from scipy.special import softmax
```

Deep learning

```
import torch
from torch import nn
```

#Huggingface

```
from huggingface_hub import notebook_login
```

✅ *Tip: It is good practice to install and import all libraries you need in one cell at the beginning of the notebook. This makes the notebook more readable, maintainable and prevents runtime errors later on.*

I have categorized all my imports into sections representing their use. These are all the libraries I need to preprocess the dataset, build, train, and evaluate a sentiment analysis model based on the tweet data, and ultimately deploy it on the Hugging Face spaces platform.

Connect to Google Drive

You can access your google drive files directly from a Colab notebook.

My dataset is stored in Google Drive. I need to mount the drive to read and write files directly from the Colab environment.

Copy

```
# Connect to your google drive  
  
drive.mount('/content/drive')
```

After running this code, you'll be prompted to authenticate your Google account to grant the notebook access to your drive.

Once authorized, you can then access files and folders from your Google Drive using the specified mount path (/content/drive).

Skip this step if you want to load the dataset with another method.

Load Dataset

Copy

```
df = pd.read_csv('/content/drive/MyDrive/Sentiment Analysis NLP/Dataset/Coachella')
```

This specifies the exact file path pointing to the location of the dataset, reads the `.csv` file into a pandas DataFrame called `df`.

The dataset can be found on [Kaggle](#).

Now lets take a look at the dataset to understand its structure.

Data Understanding

We want to quickly get a snapshot of our DataFrame.

Copy

```
df.head()
```

This will return the first 5 rows of the DataFrame.

1	positive	yes	MissTaraaa10	2	RT @sudsybuddy: WMT THIS IS ABSOLUTE FIRE_UO...	NaN	1/7/15 15:02	5.529630e+17	united states	NaN
2	positive	yes	NMcCracken805	0	#Coachella2015 #VIP passes secured! See you th...	NaN	1/7/15 15:01	5.529630e+17	Costa Mesa, CA	NaN
3	positive	yes	wopnfm	1	Philly🇺🇸 @warondrugsjams will play #Coachell...	NaN	1/7/15 15:01	5.529630e+17	Philadelphia, PA and Worldwide	Quito
4	positive	yes	Caesears	0	If briana and her mom out to #Coachella2015 L...	NaN	1/7/15 15:00	5.529630e+17	NaN	NaN

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

First 5 rows of DataFrame

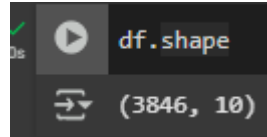
The dataset consists of the following columns:

Column Name	Description
`coachella_sentiment`	Tweet sentiment: positive, negative, neutral
`coachella_yn`	Whether the tweet mentions "Coachella" (yes/no)
`name`	Username or ID associated with individual tweets.
`retweet_count`	Number of retweets
`text`	The tweet content
`tweet_coord`	Geo-coordinates (if available)
`tweet_created`	Timestamp (All tweets are from Jan 7, 2015)
`tweet_id`	Unique ID
`tweet_location`	User's location
`user_timezone`	User's timezone

[Get the data](#) • Created with [Datawrapper](#)

Lets also get the dimensions of the DataFrame.

```
df.shape()
```



DataFrame Dimensions

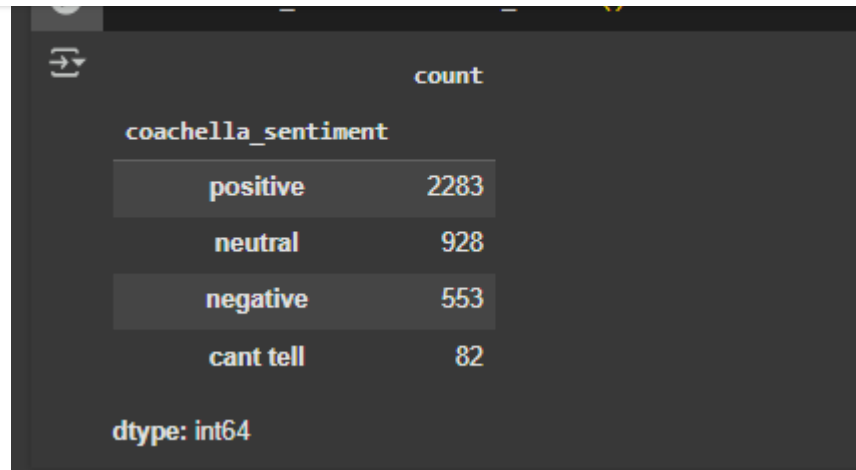
The DataFrame has 3846 rows and 10 columns.

Now lets understand the sentiment distribution.

Out of 3846 tweets we want to count how many tweets express each type of sentiment (positive, negative, neutral, or perhaps others) from the `coachella_sentiment` column within the dataset.

Copy

```
df['coachella_sentiment'].value_counts()
```



coachella_sentiment	count
positive	2283
neutral	928
negative	553
cant tell	82

dtype: int64

Sentiment Column Value Counts

A huge majority of the tweets expressed have a positive sentiment, followed by neutral and then negative sentiments. There are also a few tweets with unknown sentiment (`cant tell`).

Lastly, lets look at a sample of tweets from the `text` column.


```

text
2597 Drake head lining. #Coachella2015
3516 RT @MargsSheWrote: If you're going to #Coachella2015, just admit now you're going to do drugs in the desert. Cause everyone knows you're no
2683 RT @crisdacat: Check out the #Coachella2015 line up, music from @venusamormusic & a mash up from #luda all on\nhttps://t.co/1FNCngBr0e\n#word
3647 Really bummed Michael Bolton isn't on the #Coachella2015 lineup.
2457 #Coachella2015 tickets on sale today at 3pm EST. Full lineup: http://t.co/WAwRprsg72 http://t.co/FGwX4Awk7p
2302 #Coachella2015 because @Interpol and @brandnewrock.
1656 @2015Coachella: Stay up to date with all the latest news for Coachella 2015! #Coachella #2015 #coachella2015
3175 @AZEALIABANKS will you really be at #coachella2015 ???
7 #Coachella2015 I absolutely can NOT wait. This weekend is about to exceed EPIC EXPECTATIONS!!! :) Lineup is CRAZY! http://t.co/7CQXKCprLH
2203 #Coachella2015 lineup- nothing compared to last year, but still good and still going. #Countdown starts now
dtype: object

```

Sample of Tweets from text column

We can see that the tweets contain a lot of non alphabetic characters like mentions, hashtags, URLs and inconsistencies with casing and punctuation, which is expected for tweets. We will need to clean this.

While the dataset includes a lot of columns with useful information, for sentiment analysis we'll only use:

- text (the tweet)
- coachella_sentiment (the labels)

This also means we don't have to do any in-depth data understanding. Most of the columns are object columns, therefore no deep statistical analysis is needed.

Lets move on and prepare the data for EDA and modelling.

Data Preparation

After getting a basic understanding of our dataset, we perform the following actions to clean and prepare our dataset:

1. Drop unnecessary columns

Copy

```
cols_to_drop = [  
    'coachella_yn', 'name', 'retweet_count', 'tweet_coord',  
    'tweet_id', 'user_timezone', 'tweet_location', 'tweet_created'  
]  
  
df = df.drop(columns=cols_to_drop)
```

This leaves our DataFrame with our desired columns (`coachella_sentiment` and `text`).

2. Rename `coachella_sentiment` column to `labels`

```
df.rename(columns={'coachella_sentiment': 'labels'}, inplace=True)
```

It is good practice to rename this column. In deep learning (especially with Hugging Face), the target column must be named `labels` because the training step and loss computation expects the target variable to be in a column named `labels`.

3. Delete rows with `cant tell` sentiment values from the `coachella_sentiment` column.

We are only interested with the positive, neutral and negative sentiment.

Copy

```
df = df[df['labels'] != 'cant tell']
```

Since the values with a `cant tell` value are few, we can just drop them.

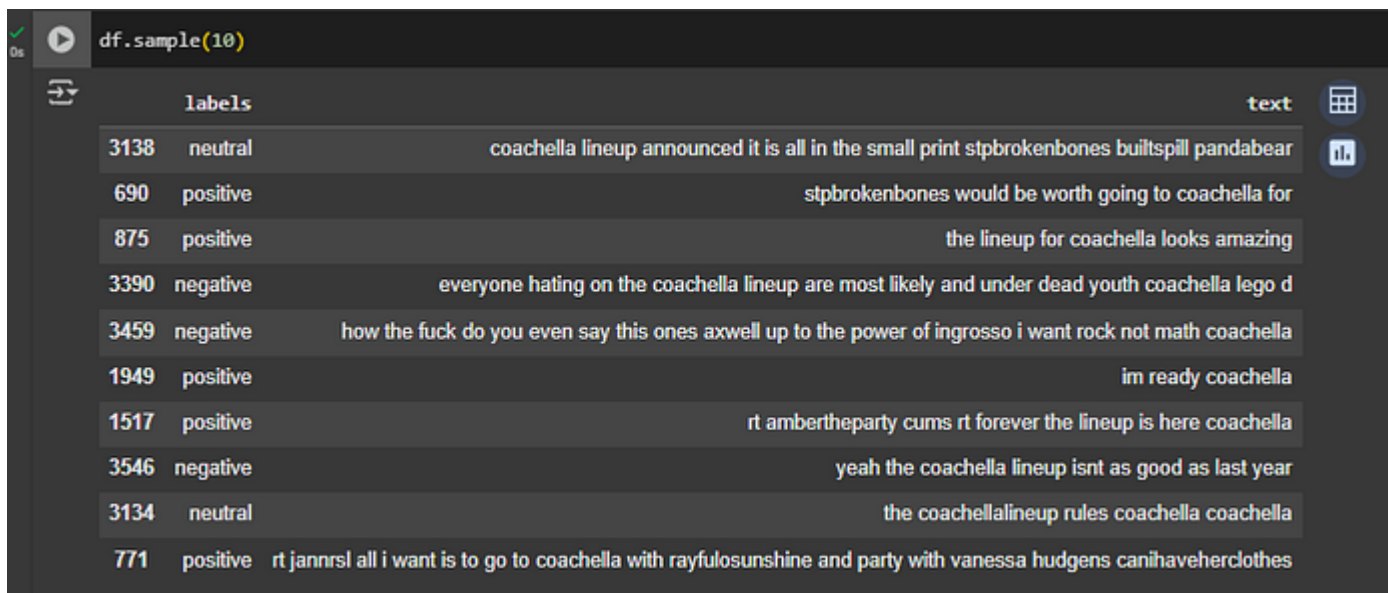
4. Clean `text` column of Twitter Handles, HTML characters, URLs and other non alphabetic characters.

We define a function to clean the tweets and apply it to the `text` column.

```
def clean_text(text):  
    # Convert text to lower case  
    text = text.lower()  
  
    # Remove URLs  
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)  
  
    # Remove tweet mentions  
    text = re.sub(r'<user>', '', text)  
    text = re.sub(r'<url>', '', text)  
  
    # Remove special characters  
    text = re.sub(r'^a-zA-Z\s]', '', text)  
  
    # Replace all whitespace characters with a single space  
    text = re.sub(r'\s+', ' ', text)  
  
    # Remove leading and trailing whitespace  
    text = text.strip()  
  
    return text  
  
df['text'] = df.text.apply(clean_text)
```

This step is important because the text is inconsistent and may affect model performance. This ensures each tweet is clean, lowercase, and free from links,

Lets look at a sample of how our DataFrame looks like now after cleaning.



```
df.sample(10)
```

	labels	text
3138	neutral	coachella lineup announced it is all in the small print stpbrokenbones builtspill pandabear
690	positive	stpbrokenbones would be worth going to coachella for
875	positive	the lineup for coachella looks amazing
3390	negative	everyone hating on the coachella lineup are most likely and under dead youth coachella lego d
3459	negative	how the fuck do you even say this ones axwell up to the power of ingrosso i want rock not math coachella
1949	positive	im ready coachella
1517	positive	rt ambertheparty cums rt forever the lineup is here coachella
3546	negative	yeah the coachella lineup isnt as good as last year
3134	neutral	the coachellalineup rules coachella coachella
771	positive	rt jannrsl all i want is to go to coachella with rayfulosunshine and party with vanessa hudsons canihaveherclothes

The tweets are much more uniform and readable now. This is much better for models to process and understand.

Now we can visualize the data before model building.

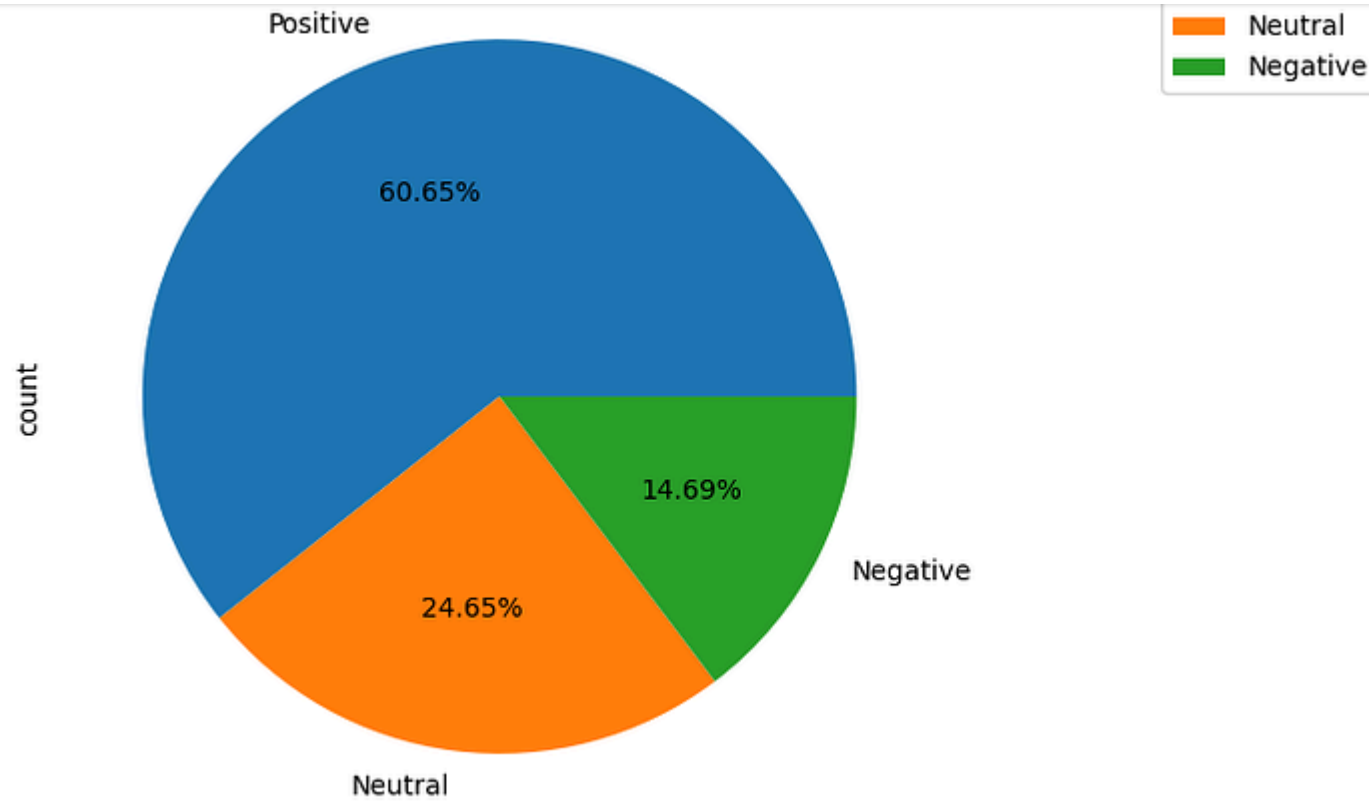
Exploratory Data Analysis

Sentiment Label Distribution (Pie Chart)

Lets visualize the proportion of our sentiment labels with a pie chart.

Copy

```
plt.figure(figsize=(6,6))  
explode=0.1,0  
df.labels.value_counts().plot.pie(autopct='%1.2f%%', labels=['Positive', 'Neutral', 'Negative'])  
plt.legend(bbox_to_anchor=(1.5,1))  
plt.show()
```



Pie chart of sentiment labels column

From the pie chart, we can see that:

- ~60% of tweets are **positive**
- ~25% are **neutral**
- ~15% are **negative**

Most Common Words (Word Cloud)

Next, lets look at the most frequent words used across all tweets. We can do this with a word cloud.

Copy

```
all_data = df['text'].to_string()
wordcloud = WordCloud().generate(all_data)
plt.figure(figsize=(12,8))
plt.imshow(wordcloud,interpolation='bilinear')
plt.title('Most Common Words')
plt.axis("off")
```




This is a great way to see the most used words in our tweets.

Tweet Length Distribution (Histogram)

line.

Copy

```
text_lengths = df['text'].str.split().str.len()

# Calculate the average
average_length = np.mean(text_lengths)

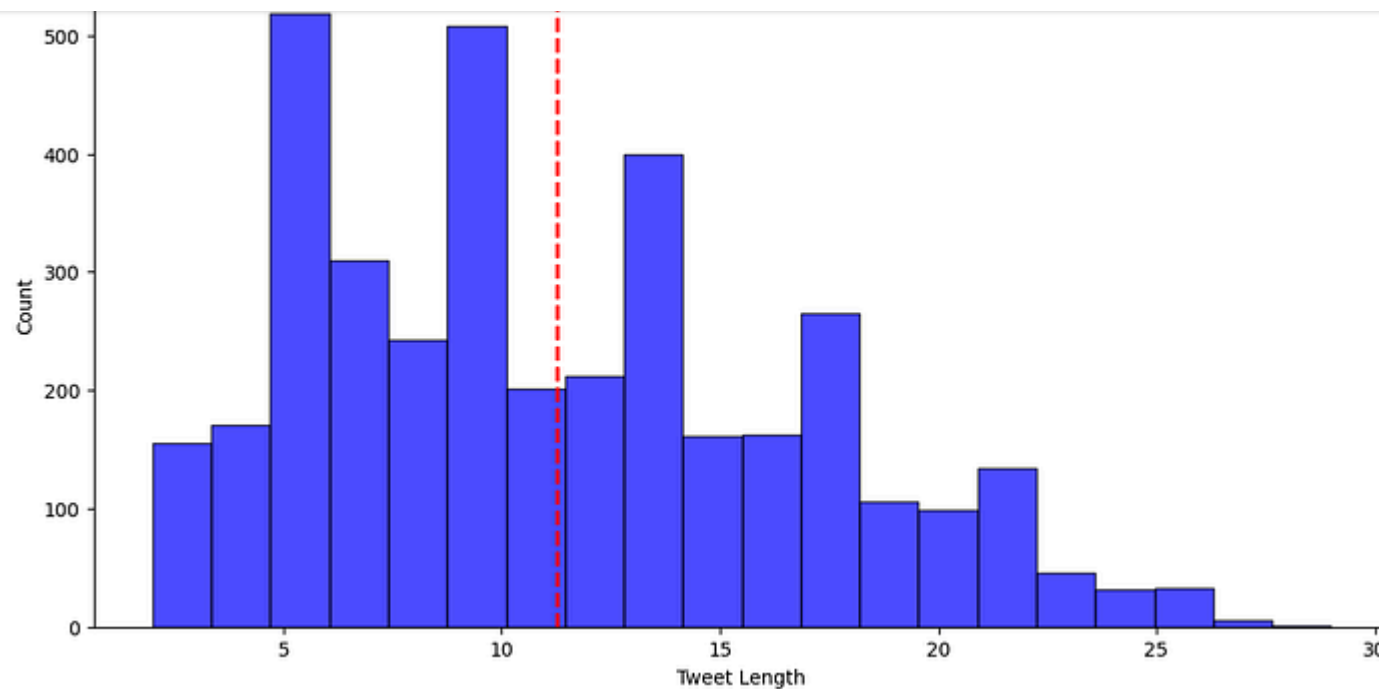
# Create a figure and axis
fig, ax = plt.subplots(figsize=(12, 6))

# Using plt.hist to create a histogram with Matplotlib
ax.hist(text_lengths, bins=20, color="blue", edgecolor="black", alpha=0.7)

# Add average line
ax.axvline(average_length, color='red', linestyle='dashed', linewidth=2, label=

ax.set_title('Histogram of Tweet Lengths')
ax.set_xlabel('Tweet Length')
ax.set_ylabel('Count')

# Display the plot
plt.show()
```



Histogram of Tweet Length and Count

Most tweets are around 5–15 words, with the **average length being ~11 words**. This short format is typical of tweets and suggests we may not need complex models for long text sequences.

Next, lets move on to creating our Model.

Modelling

Train-Test Split

First we start with splitting the cleaned dataset into training and test sets. We do this to ensure that the model's performance is evaluated on the test data it has not seen during training

Copy

```
train, eval = train_test_split(  
    df,  
    test_size=0.2,  
    random_state=42,  
    stratify=df['labels']  
)
```

This splits our DataFrame into 80% training set and 20% test set, saved in the **train** and **eval** datasets respectively.

We set a `random_state` value so we can get the same split every time we run the code.

training and evaluation sets.

Login to Hugging face Hub

We now need to login to the hugging face hub so we interact with our hugging face account. This is necessary for tasks such as:

- Accessing private models or datasets from the Hub.
- Pushing our trained models and datasets to the Hub.

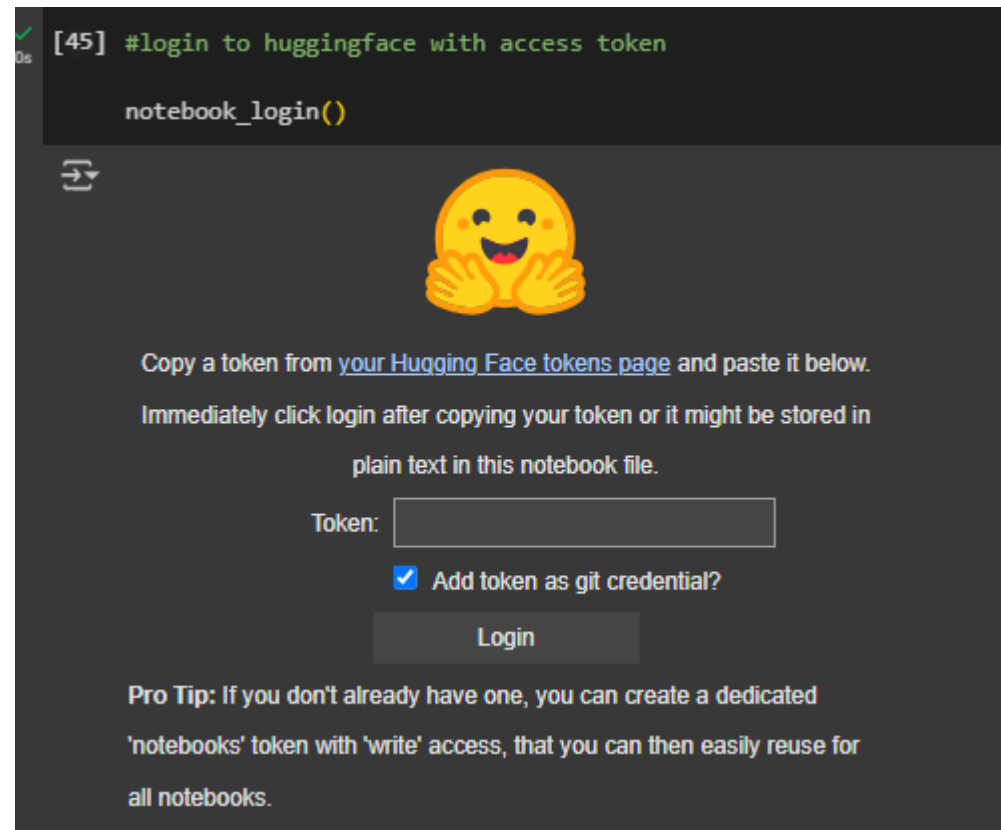
For this tutorial, we'll use [cardiffnlp/twitter-roberta-base-sentiment-latest](#) , a RoBERTa-base model trained on ~124M tweets from January 2018 to December 2021, and finetuned for sentiment analysis.

After logging in, we will be able to load our trained model and the tokenizer.

Copy

```
notebook_login()
```

This function from the `huggingface_hub` library initiates an authentication process that allows us to connect to the Hugging Face Hub.



```
[45] #login to huggingface with access token

notebook_login()
```

Copy a token from [your Hugging Face tokens page](#) and paste it below.
Immediately click login after copying your token or it might be stored in plain text in this notebook file.

Token:

☒ Add token as git credential?

Login

Pro Tip: If you don't already have one, you can create a dedicated 'notebooks' token with 'write' access, that you can then easily reuse for all notebooks.

Login to Hugging face Hub

You can obtain a token from your [Hugging Face](#) account settings. This token provides secure access to your account and allows you to perform various actions.

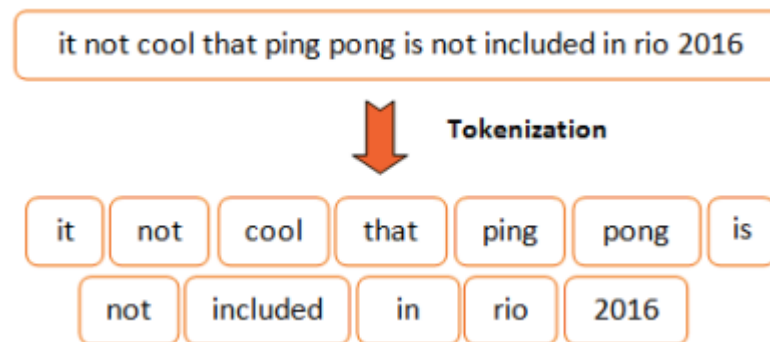
👉 Check out [this video](#) on how to generate a Hugging Face access token.

Tokenization

Tokenization in sentiment analysis is the process of converting raw text into smaller parts called *tokens*.

You can think of tokens as parts to a whole like:

- A word is a token in a sentence
- A sentence is a token in a paragraph.



Tokenization in NLP | [Source](#)

In our case, we want to segment our tweets into words and sub words, treating them as different elements. This helps models handle rare or unknown words by breaking them into known pieces.

Remember that computers only understand 0's and 1's, not words. So before feeding text data into a model, tokenization is an essential step.

Copy

```
roberta = f"cardiffnlp/twitter-roberta-base-sentiment-latest"  
tokenizer = AutoTokenizer.from_pretrained(roberta)
```

The above code does the following:

- Defines a variable `roberta` and assigns it the string `"cardiffnlp/twitter-roberta-base-sentiment-latest"`. This string is the link to the trained model we are using from huggingface.
- Uses the `AutoTokenizer.from_pretrained()` method to load the correct tokenizer associated with our model, defined in the `roberta` variable. The `AutoTokenizer` class is convenient because it figures out the appropriate tokenizer class based on the model name. The loaded tokenizer is then stored in the `tokenizer` variable.

Our tokenizer will do the following:

- **Converts those tokens into numbers** Each token maps to a unique number from the model's vocabulary. e.g. ["un", "##happiness"] → [2346, 4782] (*token IDs*)

After the tokenizer is loaded in the notebook, we can tokenize our text data.

First we define a function to tokenize our text.

Copy

```
def tokenize_data(example):  
    return tokenizer(example['text'], max_length = 128, padding='max_length', t
```

This function takes our loaded `tokenizer` to process the text data with a few parameters:

- `max_length=128` : This breaks down each text into exactly 128 tokens. No more and no less. Most BERT models have a limit of 512 tokens. 128 is often used to speed up training and save memory when sentences are short like with tweets.
- `padding='max_length'` : If a text is shorter than 128 tokens, then extra padding (tokens) will be added to make it the length equal to `max_length` which is 128

- `truncation=True` . If a text is longer than the specified `max_length` which is 128 tokens, it should be cut off or trimmed down.

Next, we need another function to transform our sentiment labels from their original text format ('negative', 'neutral', 'positive') into numerical representations (0, 1, and 2 respectively) that are suitable for modelling.

Copy

```
def transform_labels(labels):  
  
    labels = labels['labels']  
    num = 0  
    if labels == 'negative':  
        num = 0  
    elif labels == 'neutral':  
        num = 1  
    elif labels == 'positive':  
        num = 2  
  
    return {'labels': num}
```

This function takes a dictionary containing the text label, extracts the label string, and then uses conditional logic to assign an integer value based on the sentiment, returning a new dictionary with the numerical label.

Next, we then convert our training and evaluation pandas DataFrames into Hugging Face Dataset objects.

Copy

```
train_dataset = Dataset.from_pandas(train)
eval_dataset = Dataset.from_pandas(eval)

dataset = DatasetDict({
    'train': train_dataset,
    'eval': eval_dataset
})
```

We do this because the Hugging Face Trainer class, which will be used for model training, expects data in the Dataset format. We then organize the datasets into a dataset dictionary, under the variable dataset .

Now we can tokenize our text.

Copy

```
remove_columns = ['labels', 'text']  
dataset = dataset.map(transform_labels, remove_columns=remove_columns)
```

We apply the tokenization function to each item in the `dataset` object. The `batched=True` argument means that the function is applied to batches of data, which is generally more efficient for processing large datasets.

We also convert the original sentiment labels to numerical format by applying the `transform_labels` function to the `dataset` object.

At the same time, we remove the original columns (`'labels'` and `'text'`) as they are no longer needed after tokenization. However, we will have a `labels` column because the original text labels are replaced by the numerical labels generated by the `transform_labels` function.

Lets inspect the structure and contents of our processed data after the tokenization and column removal steps.

```
DatasetDict({
  train: Dataset({
    features: ['labels', '__index_level_0__', 'input_ids', 'attention_mask'],
    num_rows: 3011
  })
  eval: Dataset({
    features: ['labels', '__index_level_0__', 'input_ids', 'attention_mask'],
    num_rows: 753
  })
})
```

Tokenized Training and Evaluation Dataset Objects

We now have clean and tokenized training and evaluation datasets. The result of the tokenization process includes the following columns/features:

- `labels` : Transformed numerical labels column (0, 1, 2).
- `__index_level_0__` : This is an index column created from the conversion of our pandas DataFrames into dataset objects. We can delete this.
- `input_ids` : These are the **token IDs** for each tweet.
- `attention_mask` : These tell the model which tokens are real (1) and which are padding (0).

Now we can move on to balancing our target classes.

Balancing Target Classes

Since our target has imbalanced class weights (positive, neutral and negative don't have an equal number of samples), we want to give more weight to underrepresented classes and give less weight to classes with more samples.

Copy

```
labels = dataset['train']['labels']  
  
class_weights = compute_class_weight('balanced', classes=np.unique(labels), y=
```

First we need to extract the labels from the `labels` column for all samples in the `train` split of the `dataset` object.

We then compute the weights for each label class in the train data with the `compute_class_weight` function. The `balanced` argument tells the function to calculate weights inversely proportional to the class frequencies in our labels. This means classes with fewer samples will be assigned higher weights, and classes with more samples will receive lower weights.

We can preview if this is true after the computation.

```
class_weights, np.unique(labels)  
(array([2.26561324, 1.35265049, 0.54965316]), array([0, 1, 2]))
```

Class Weight Computation

This has been successful. The negative class (0) now has a higher weight and the positive class (2) has a lower class weight. The neutral class (1) has a slightly higher weight as well.

We are now ready to initialize and fine tune our pre-trained model.

Initialize pre-trained model

Let's define an instance of the pre-trained model with the number of output classes it should have (3).

Copy

```
model = AutoModelForSequenceClassification.from_pretrained(roberta, num_labels=
```

We use the `AutoModelForSequenceClassification` class from the `transformers` library to load the pre-trained model we saved earlier in the `roberta` variable.

tuning is the process of continuing the training of a pre-trained model on a **task-specific dataset**, allowing it to adapt to the nuances of your domain.

Configure training parameters and evaluation metric

Now that we have our model instance, we must configure the parameters for training the model.

Copy

```
training_args = TrainingArguments("./results",  
    report_to = "none"  
    num_train_epochs=3,  
    load_best_model_at_end=True,  
    eval_strategy='epoch',  
    save_strategy='epoch',  
)
```

We give the output directory a name `./results` where the model and other logs will be saved. We also set these parameters:

- `report_to = "none"` : This determines where the training progress and results are reported. Hugging Face automatically integrates with W&B for experiment tracking and visualization. It can be useful but it is not necessary.

- `num_train_epochs=3` : Epochs are the number of times the model will repeat the training loop over the training dataset. In this case, the model will be trained for 3 epochs.
- `load_best_model_at_end=True` : After training is complete, the model with the best performance on the evaluation dataset is loaded.
- `eval_strategy='epoch'` : The model's performance will be evaluated at the end of each training epoch.
- `save_strategy='epoch'` : The model's state will be saved at the end of each training epoch.

Next, we need to configure how we will evaluate the performance of our model during training.

Copy

```
metric = load("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return metric.compute(predictions=predictions, references=labels)
```

Accuracy calculates the proportion of correctly predicted labels out of the total number of predictions. It is a common evaluation metric in classification tasks, especially with a balanced dataset.

We define a function `compute_metrics` which will extract the raw output scores (`logits`) and the actual labels (`labels`) from the evaluation predictions, and then uses `np.argmax` to find the index of the highest score among the raw scores to get the predicted class for each data point. Finally, the accuracy is calculated by comparing the predictions and the actual labels. This function will be called automatically during training and evaluation, at the end of each epoch, to give us an overall accuracy score.

Define Custom Trainer

We're almost there.

To train our model, we will use the `trainer` class from the `transformers` library.

However, we want to add our own custom class weights that we calculated to address imbalances. So we can create a custom class by using the original `trainer` class and then override its default settings to edit the loss function.

```
class CustomTrainer(Trainer):
    def compute_loss(self, model, inputs, return_outputs=False, num_items_in_batch=None):
        labels = inputs.get("labels")

        # Forward pass
        outputs = model(**inputs)
        logits = outputs.get("logits")

        # Apply Class Weights
        class_weights_tensor = torch.tensor(class_weights, dtype=torch.float32)

        # Compute custom loss
        loss_fct = nn.CrossEntropyLoss(weight=class_weights_tensor)
        loss = loss_fct(logits.view(-1, self.model.config.num_labels), labels.view(-1))
        return (loss, outputs) if return_outputs else loss
```

This code defines a custom loss function using `nn.CrossEntropyLoss` to handle class imbalance by applying our class weights within the `compute_loss` method of our custom trainer class.

The `compute_loss` method first extracts the true labels (`labels`) from the inputs, runs the model on the inputs to get predictions (`outputs`) and extracts the raw prediction scores (`logits`) from the model outputs.

GPU). Since the loss function is a PyTorch operation, it expects all its inputs including weights to be in PyTorch tensor format.

These weights are then passed into the loss function (`nn.CrossEntropyLoss`). Our loss function will now apply different importance (weights) to each class.

The weighted loss between predicted logits and true labels are calculated and finally, the method returns either just the loss or both the loss and model outputs, depending on whether `return_outputs` is set to `True` . In our case, it is set to `False` , so we only get the loss.

Now we are ready to fine-tune our model.

Training and Evaluation

Finally we can train our model with our data.

First we select and define our training and test datasets from our dataset object.

Copy

We select the training and test splits and randomly shuffle the data using the same seed (10) to get the same result each time we run the code.

Next we define an instance of our custom trainer class (CustomTrainer).

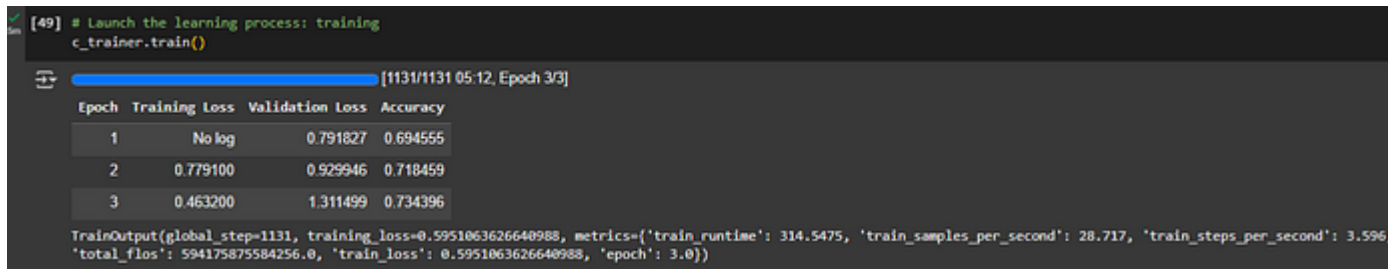
Copy

```
c_trainer = CustomTrainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_dataset,  
    eval_dataset=eval_dataset,  
    tokenizer = tokenizer,  
    compute_metrics=compute_metrics,  
)
```

The custom trainer class takes our model, our training parameters, train and test datasets, our tokenizer and evaluates using our accuracy metric.

Our custom trainer is now ready to be used for training.

Copy



```
[49] # Launch the learning process: training
c_trainer.train()
```

Epoch	Training Loss	Validation Loss	Accuracy
1	No log	0.791827	0.694555
2	0.779100	0.929946	0.718459
3	0.463200	1.311499	0.734396

TrainOutput(global_step=1131, training_loss=0.5951063626640988, metrics={'train_runtime': 314.5475, 'train_samples_per_second': 28.717, 'train_steps_per_second': 3.596, 'total_flos': 594175875584256.0, 'train_loss': 0.5951063626640988, 'epoch': 3.0})

Model Training Results

Training and Evaluation Summary

Our model has successfully trained across 3 epochs. Lets look at the key points:

- The **training loss** measures how well the model is performing on the training data. The lower this gets the better. Training loss dropped significantly from 0.7791 → 0.4632. This means the model is learning the training data better over time.
- The **validation loss** tells us how well the model performs on the test set, which is unseen data. This is calculated using the same loss function but after each epoch. Validation loss increased from 0.7918 → 1.3115. This is a sign of **overfitting**, the most common issue in data science and machine learning.
- The **accuracy** is our evaluation metric, measuring percentage of correct predictions. Accuracy improved slightly from 69% → 73%, which is a positive

Overall, our model is learning patterns in the training data well, but validation loss is increasing as training loss is decreasing, which is a sign of overfitting. This means our model is memorizing the training data but can't generalize to new unseen data.

Overfitting is like when you cram before a test, but you don't understand the topic you are studying. When the test comes, you do great if the questions are exactly the same but struggle if the questions are phrased differently.

Lets do a final evaluation of our model.

Copy

```
c_trainer.evaluate()
```

```
# Launch the final evaluation
c_trainer.evaluate()

[95/95 00:05]
{'eval_loss': 0.7918271422386169,
 'eval_accuracy': 0.6945551128818062,
 'eval_runtime': 5.5263,
 'eval_samples_per_second': 136.257,
 'eval_steps_per_second': 17.19,
 'epoch': 3.0}
```

These results are the final evaluation metrics after the 3rd epoch of training. They show us how well the fine-tuned model performs on unseen data (test set).

An eval loss of 0.79 and accuracy of 69.5% is decent. However there is room to improve generalization, since the training accuracy is significantly higher than eval accuracy.

The main culprit of overfitting is usually a small or imbalanced dataset. In our case, we already addressed class imbalance using a weighted loss function. The next step would be to source more diverse real world data or artificially increase the size and diversity of the dataset. More data points make it harder for the model to memorize specific examples and help it generalize better on new unseen data.

There are other numerous ways to address overfitting, like early stopping and regularization, but those are more advanced topics.

For now, our model's accuracy is good enough for inferencing.

Push model to Hugging Face Hub

explore, collaborate and experiment with open-source Machine Learning.

It's completely open source, meaning anyone can view, use, or build upon your work.

Uploading is simple and only takes a few lines of code.

Copy

```
model.push_to_hub("Azie88/Coachella_sentiment_analysis_roberta")
tokenizer.push_to_hub("Azie88/Coachella_sentiment_analysis_roberta")
dataset.push_to_hub("Azie88/Coachella_sentiment_analysis_roberta")
```

Since my notebook is already authenticated with my hugging face account, the files will be pushed directly to my namespace `Azie88` on the Hub.

Inferencing

Now lets try out our model and make predictions with real world unseen tweets.

Load the Fine-Tuned Model from Hugging Face Hub

Copy

```
model_path = f"Azie88/Coachella_sentiment_analysis_roberta"

tokenizer = AutoTokenizer.from_pretrained(model_path)
config = AutoConfig.from_pretrained(model_path)
model = AutoModelForSequenceClassification.from_pretrained(model_path)
```

Just like we previously loaded a pre-trained model and tokenizer, here we're loading our own fine-tuned version that we just trained and saved to the Hub.

We are loading the model and its associated tokenizer, and the configuration which contains metadata such as the number of output labels (in this case 3) and other architectural details.

Preprocess Input Tweets

Next we define a function, similar to the one we used to clean our dataset text, to clean and standardize the input tweets for prediction.

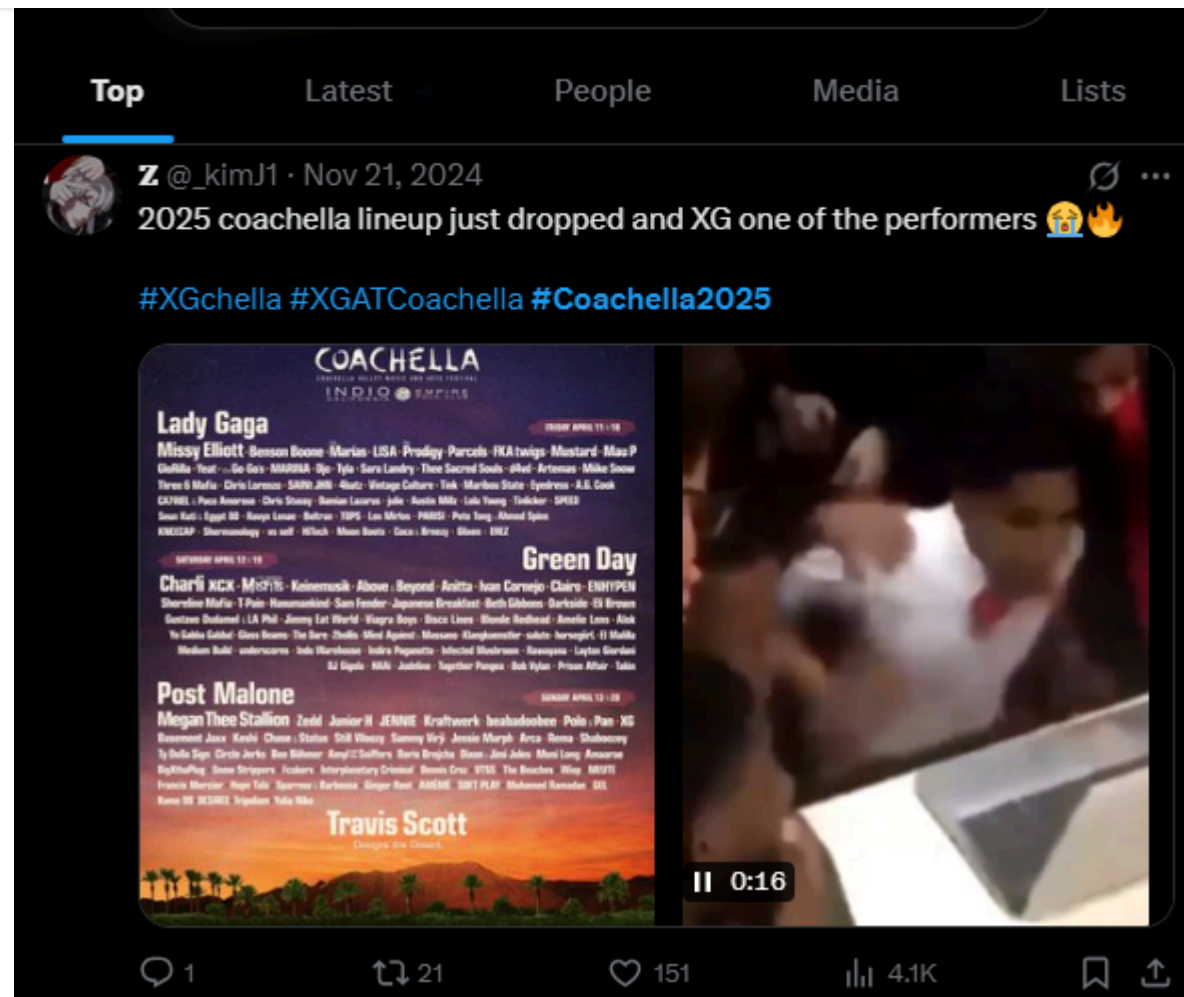
Copy

```
for t in text.split(" "):  
    t = '@user' if t.startswith('@') and len(t) > 1 else t  
    t = 'http' if t.startswith('http') else t  
    new_text.append(t)  
return " ".join(new_text)
```

This function will split the input text into individual words. It then replaces any user mentions (e.g. @someone) with a generic placeholder @user , and also replaces any URLs with a generic placeholder http . This keeps the model input consistent and reduces noise.

Sample a Real Tweet

Now let's find a tweet from **Coachella 2025** on X to test.



I did a custom search to look for tweets posted within 2 months after the Coachella lineup was released.

Lets input this tweet and make a prediction.

```
text = "2025 Coachella lineup just dropped and xo one of the performers 🙌🔥 #"  
text = preprocess(text)
```

This code takes the tweet text and passes it through our `preprocess` function to clean it. While this specific tweet doesn't contain any `@user` mentions or URLs, the function ensures all inputs are standardized just in case.

Predict Sentiment

Now, let's tokenize the cleaned tweet before passing it through our fine-tuned sentiment analysis model.

Copy

```
# Tokenization and Class Prediction  
encoded_input = tokenizer(text, return_tensors='pt')  
output = model(**encoded_input)  
scores = output[0][0].detach().numpy()  
scores = softmax(scores)
```

The above code will tokenize the tweet text and return as PyTorch tensors, which is what the model expects. It then runs the text through the model to get raw

Lastly, we map those prediction scores to sentiment labels we can read.

Copy

```
config.id2label = {0: 'NEGATIVE', 1: 'NEUTRAL', 2: 'POSITIVE'}

# Print labels and scores
ranking = np.argsort(scores)
ranking = ranking[::-1]
for i in range(scores.shape[0]):
    l = config.id2label[ranking[i]]
    s = scores[ranking[i]]
    print(f"{i+1}) {l} {np.round(float(s), 4)}")
```

We define a dictionary to map of class numbers to text sentiment labels. The scores are then sorted in descending order (highest score first), and the predicted labels with their respective confidence scores are printed.

```
# Print labels and scores
ranking = np.argsort(scores)
ranking = ranking[::-1]
for i in range(scores.shape[0]):
    l = config.id2label[ranking[i]]
    s = scores[ranking[i]]
    print(f"{i+1}) {l} {np.round(float(s), 4)}")
```

```
➡ 1) POSITIVE 0.8819
   2) NEUTRAL 0.104
   3) NEGATIVE 0.0141
```

Tweet Sentiment Prediction Confidence Scores

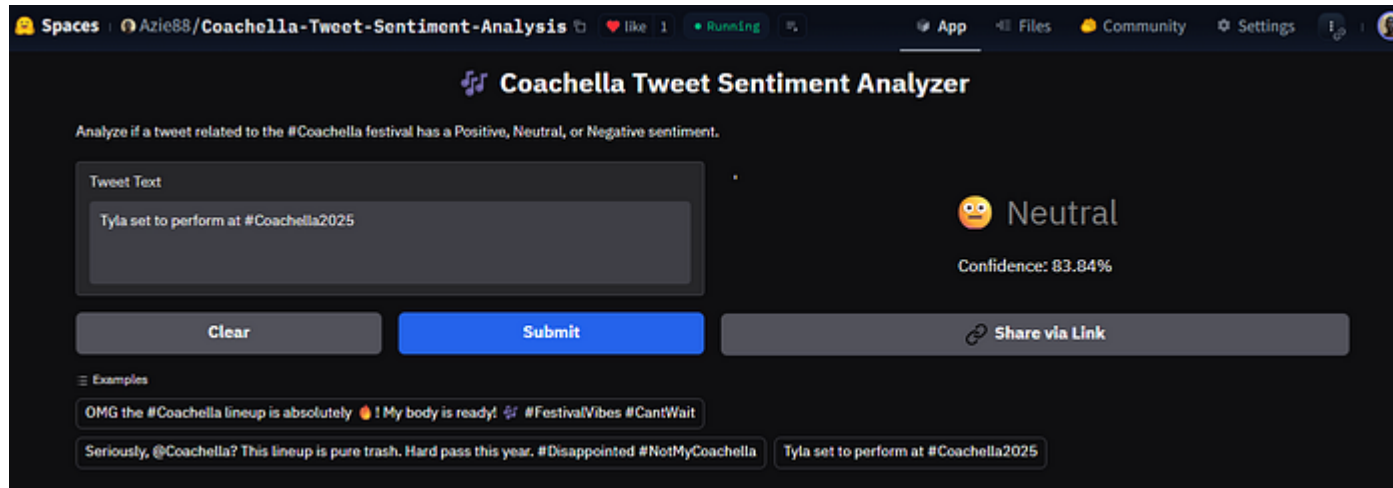
Our tweet has a positive sentiment with a confidence score of 88%. This makes sense, as the user expresses excitement about the lineup reveal.

Our model fine-tuning is successful and is working as expected! 🔥

Deploying Gradio App

Now that we have a working fine tuned model, we can create a user interface to we can interact with it freely.

I've built a **Gradio app** that uses the same inferencing process.



Coachella Tweet Sentiment Analysis Gradio App on Hugging Face Spaces

👉 You can [check out the app and interact with it here](#).

Building apps with Gradio is very easy. If you'd like to learn how, check out my tutorial:

Deploying a ML Model with Gradio and Hugging face — Python Machine Learning

How to build a Gradio Web App with a Machine Learning Model and put it on Hugging Face spaces

Final Thoughts

This project shows how powerful and accessible it is to fine-tune deep learning models for real-world tasks like sentiment analysis on real-world social media content. You can do it all open source too.

A practical use case for this sentiment analysis model is integrating it into a larger social media monitoring system for event organizers, entertainment brands, or digital marketing teams. For example:

 *Imagine a company dashboard that automatically pulls recent tweets after a lineup or ticket announcement, runs sentiment analysis in real-time, and summarizes public reaction before the event even launches.*

Such a tool could help marketing teams gauge audience mood, detect PR risks early, and make data-informed adjustments to campaigns or logistics.

Some potential improvements that could be made are:

- **Using a Multilingual Model:** The `twitter-roberta-base-sentiment-latest` model is mainly suitable for English. To support tweets in Spanish, French,

- **Adding More Diverse Data:** While we addressed class imbalance, the model needs more data, especially from different time periods and subcultures. This would help the model generalize better and reduce overfitting.
- **Using a Multimodal Model:** Many tweets include images or videos, which can carry strong emotional expression, like memes. Text-only models like RoBERTa ignore attached media and visual sentiment. A combination of a text encoder with a vision model could possibly be explored to create a more powerful solution to predict sentiment more accurately.

References

Fine-Tuning Bert for Tweets Classification ft. Hugging Face

Bidirectional Encoder Representations from Transformers (BERT) is a state of the art model based on transformers...

medium.com

Posted by Jakob Uszkoreit, Software Engineer, Natural Language Understanding
Neural networks, in particular recurrent...

[research.google](#)

Getting Started with Sentiment Analysis on Twitter

We're on a journey to advance and democratize artificial intelligence through open source and open science.

[huggingface.co](#)

What Is NLP (Natural Language Processing)? | IBM

Natural language processing (NLP) is a subfield of artificial intelligence (AI) that uses machine learning to help...

[ibm.com](#)

NLP Techniques: 7 Tasks With Python for Better Understanding

Freedium

ko-fi librepay patreon

datasciencedojo.com

Thank you for reading!

You can access all the code for this project in my [github repository](#). Show your appreciation if you found this insightful. Leave me a comment if you have any points or suggestions.

Connect with me on [LinkedIn](#).

[#deep-learning](#)[#python](#)[#sentiment-analysis](#)[#neural-networks](#)[#data-science](#)